

INTRODUZIONE ALLA PROGRAMMAZIONE AD OGGETTI

(OBJECT ORIENTED PROGRAMMING – OOP)

PARADIGMA DI PROGRAMMAZIONE = MODELLO DI RIFERIMENTO

Un paradigma di programmazione è uno stile specifico, costituito da un insieme di regole e funzionalità, al quale il programmatore fa riferimento nell'affrontare, e quindi risolvere, uno specifico problema di natura informatica.

PARADIGMA OOP: il paradigma OOP prevede la definizione di oggetti software che comunicano tra loro attraverso lo scambio di messaggi.

Il paradigma OOP è stato introdotto a livello di ricerca alla fine degli anni '70 per via di limitazioni che il paradigma procedurale presentava: si volevano costruire oggetti troppo grandi e complessi rispetto agli strumenti che si avevano a disposizione.

Perché si studia Java?

Al giorno d'oggi realizzare applicazioni sia per desktop sia per smartphone richiede, quasi sempre, un approccio ad oggetti.

L'OOP ha fornito risposte innovative e tecnologicamente avanzate sotto il profilo tecnico-organizzativo nello sviluppo del software.

Per realizzare oggetti software via via crescenti, è necessario utilizzare strumenti adeguati a produrre oggetti di quelle dimensioni.

INGEGNERIA DEL SOFTWARE

Disciplina che cerca di fornire procedure e metodi standardizzati da applicare al processo di produzione del software. Lo scopo dell'ingegneria del software è di **pianificare, progettare, sviluppare e mantenere** il software tramite lavoro di gruppo. Tale attività ha senso per progetti di grosse dimensioni e di notevole complessità ove si rende necessaria la pianificazione.

L'OOP è una strategia che l'uomo ha inventato e sviluppato per produrre software di complessità sempre maggiore.

La crescita, in termini di complessità del prodotto finale, comporta una crescita degli strumenti utilizzati per produrlo: questo vale per tutti i prodotti industriali, software compreso.

I linguaggi di programmazione procedurale non sono adeguati a realizzare l'infrastruttura di software aventi grandi dimensioni ed elevata complessità.

L'OOP riduce i costi di produzione e di manutenzione: per produrre via via software sempre più complessi è necessario ridurre il costo di un'unità di lavoro; in caso contrario i costi diventerebbero, più o meno velocemente, ingestibili.

Cosa determina i costi di manutenzione?

ERRORS / 1K SLOC

1K SLOC = 1000 SOURCE LINE OF CODE (errori per mille righe di codice)

Per errori non si intendono errori legati al codice, ma difetti nel processo di produzione di software industriali che possono causare difetti e comportamenti non desiderati.

Perché è importante questa formula?

La formula è importante perché all'aumentare degli errori aumentano i costi di manutenzione (non in modo lineare, ma esponenziale).

SCOPO DELLA PROGRAMMAZIONE OOP

Realizzare software:

- SICURI
- RIUSABILI
- FLESSIBILI
- DOCUMENTABILI
- PORTABILI
- **ESTENDIBILI** (principio chiave della progettazione software che consente ad un sistema di essere facilmente ampliato con nuove funzionalità senza dover modificare in modo significativo il codice originale)

```
int v[20];  
int i;  
void sort(){} //ordinamento (procedura)  
int cerca(int n){} //ricerca (funzione)  
void init(){} //inizializzazione (procedura)  
int main(){  
    init(); //invocazione  
    sort(); //invocazione  
    cerca(20); //invocazione  
}
```

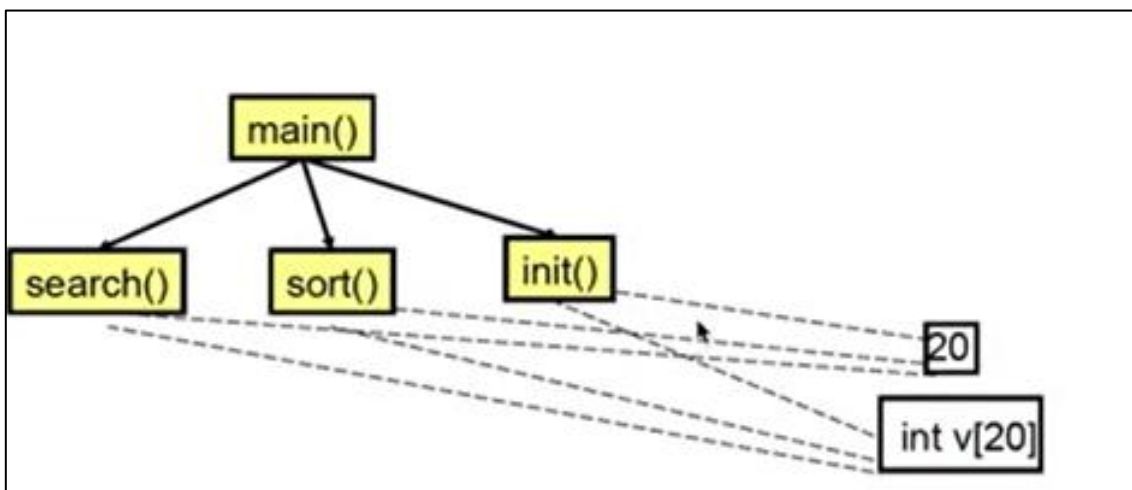
La programmazione di tipo procedurale (es. C/C++) tiene separati i dati dalle operazioni che lavorano sui dati stessi.

Le linee tratteggiate rappresentano le relazioni tra le variabili e le funzioni.

All'aumentare del numero di variabili e di funzioni, questa rete di relazioni diventa sempre più intricata.



“Spaghetti code”: codice difficile da mantenere poiché tutto è in relazione con tutto e una modifica da una parte provoca cambiamenti indesiderati da un'altra parte. Tutte le componenti sono talmente collegate le une alle altre che è molto difficile prendere un modulo (esempio una funzione) e separarlo dal resto.



L'OOP è una strategia che l'uomo ha inventato e sviluppato per produrre software di complessità sempre maggiore.

La OOP considera il software come un insieme di entità (moduli) che contengono sia i dati sia le operazioni che operano sui dati stessi. —> **MODULARITÀ DEL CODICE**

—> No relazioni di tutto con tutto. Le relazioni devono essere le più semplici possibile e ciascun modulo deve possedere un'interfaccia con la quale comunica con l'esterno.

OBJECT – ORIENTED PROGRAMMING APPROCCIO

```
class Vector{
    //internal data
    private int dim=20; //dato
    private int v[dim]; //dato
    public Vector() { //metodo per inizializzare il vettore
        for (int i=0; i<dim; i++)
            v[i]=0;
    }
    public sort() {...}
    public cerca() {...}
}
```

Le funzioni non sono separate dai dati, ma tutto quanto fa parte di un modello astratto chiamato CLASSE.

Attraverso la classe è possibile definire (istanziare) nuovi oggetti, in questo caso di tipo Vector.

```
Vector v1 = new Vector(); //istanza
```

```
Vector v2 = new Vector(); //istanza
```

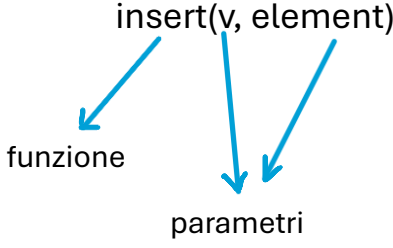
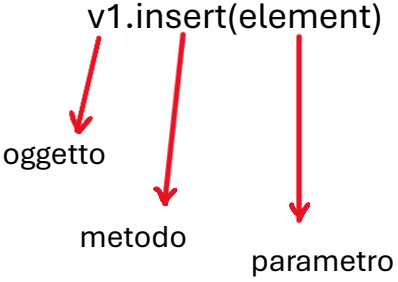
Se gli oggetti v1 e v2 necessitano di svolgere alcune operazioni, è possibile scrivere quanto segue:

```
v1.sort();
```

```
v2.cerca(22);
```

Il simbolo “ . ” serve per associare all’oggetto il metodo specifico definito all’interno della classe di appartenenza.

v1++ : non posso fare l’incremento (ERRORE). v1 non è un vettore nel senso tradizionale del termine. **v1 E’ UN OGGETTO.**

C LANGUAGE	OOP LANGUAGE
	

CLASSI, OGGETTI, ATTRIBUTI E METODI

Def1.(CLASSE): una classe rappresenta la descrizione di una specifica categoria di oggetti individuati da caratteristiche e comportamenti simili.

Esempio:

```
public Car {
```

```
    String color;
```

```
    String model;
```

```
    String brand;
```

```
    String fuel;
```

ATTRIBUTI/PROPRIETÀ/CARATTERISTICHE

Gli attributi definiscono lo stato interno di un oggetto.

//Metodo costruttore

```
Public Car(String color, String model, String brand, String fuel)
```

```
{
```

```
    this.color = color;
```

```
    this.model = model;
```

```
    this.brand = brand;
```

L’operatore this (**this.**) assegna all’attributo color il valore che si trova all’interno del parametro.

Per evitare l’ambiguità dovuta al fatto che il nome assegnato all’attributo è uguale al nome assegnato al parametro, si utilizza l’operatore **this.**

```

        this.fuel = fuel;
    }
}

```

LO SCOPO DEL METODO COSTRUTTORE È DI CREARE GLI OGGETTI APPARTENENTI AD UNA CLASSE.

Def.(**COSTRUTTORE**): il costruttore è un metodo “speciale” che ha la funzione di inizializzare gli attributi di istanza (oggetto) di una classe.

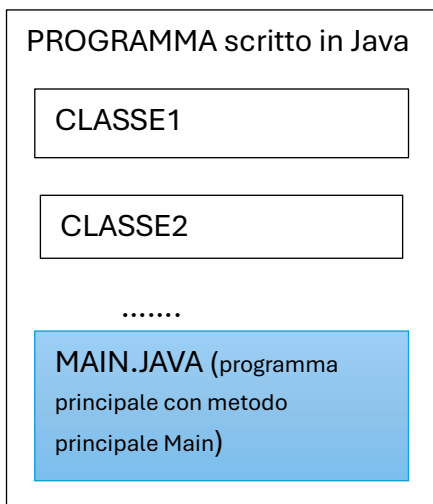
Def2.(**CLASSE**): una classe rappresenta un modello formale per la descrizione di un certo tipo di oggetti definendone:

- ATTRIBUTI
- METODI
- INTERFACCE

Def.(**ATTRIBUTI**): gli attributi sono le componenti informative (DATI) che definiscono le caratteristiche degli oggetti appartenenti ad una classe.

Def.(**OGGETTO**): per oggetti si intendono le ISTANZE materiali della classe.

Def.(**ISTANZIARE**): creare oggetti simili che condividono lo stesso insieme di attributi, lo stesso insieme di metodi e la stessa interfaccia.



Come istanzio gli oggetti appartenenti alla classe Car?

```
Car c1 = new Car(Green, Mustang, Ford, Gasoline);
```

```
Car c2 = new Car(Blue, Golf, VW, Diesel);
```

new: operatore di istanziazione

Tale operatore permette di creare una nuova istanza di un oggetto appartenente ad una determinata classe e, conseguentemente, viene allocata automaticamente nell'**heap di sistema** la memoria necessaria a conservare tale oggetto.

Conclusioni:

le classi sono categorie di oggetti; gli oggetti sono particolari istanze di quella classe, caratterizzati da uno stato interno che viene inizializzato dal metodo costruttore.

Def. (**STATO INTERNO DI UN OGGETTO**): lo stato interno di un oggetto è l'insieme dei valori delle sue proprietà in un determinato istante di tempo (**valori assunti dalle variabili/attributi**). Se cambia anche un solo valore di una proprietà di un oggetto, il suo stato interno varierà di conseguenza.